

JEONME

Panduan Developer

*Onboarding, Alur Kerja Git, dan Cara Staging &
Production Bekerja*

Dokumen	DEVELOPER-GUIDE v1.0
Tanggal	8 Juli 2026
Repo	ljeon-Corp/JeonMe
Status	v0.1.0 live di production

Daftar Isi

1. Ringkasan Proyek

2. Sebelum Mulai — Akses & Tools yang Dibutuhkan

3. Setup Development Lokal

4. Struktur Repository

5. Alur Kerja Git & Kontribusi

6. Bagaimana Staging Bekerja

7. Bagaimana Production Bekerja

8. Environment & Secrets — Ringkasan

9. Troubleshooting & Pertanyaan Umum

10. Peta Dokumentasi Lain

1. Ringkasan Proyek

Jeonme adalah platform link-in-bio + monetisasi produk digital untuk kreator Indonesia (referensi: Linktree, lynk.id). Lihat [Dokumentasi-Frontend-Jeonme.pdf](#) dan [Dokumentasi-Backend-Jeonme.pdf](#) untuk detail fitur, dan [Rencana-Sprint-Jeonme.xlsx](#) untuk backlog & prioritas kerja saat ini.

LAYER	TEKNOLOGI
Frontend	Next.js 16 (App Router) + React 19 + TypeScript + Tailwind CSS
Backend	Go 1.25 + Gin
Database	PostgreSQL 16
Cache	Redis 7
Container	Docker + Docker Compose
CI/CD	GitHub Actions
Registry image	GitHub Container Registry (GHCR)

Status saat ini: rilis pertama (`v0.1.0`) sudah live di production. Fitur inti (auth dasar, halaman publik, produk — parsial) sudah ada; checkout, saldo/penarikan, dan analitik masih dalam pengerjaan.

2. Sebelum Mulai — Akses & Tools yang Dibutuhkan

AKSES	UNTUK APA	WAJIB?
-------	-----------	--------

Kolaborator repo Ijeon-Corp/JeonMe	Push branch, buka PR	Wajib
Akses VPS (SSH)	—	Tidak untuk kerja harian — deploy sepenuhnya lewat CI/CD otomatis. Hanya pengelola infra.
GitHub Secrets/Environments	—	Tidak — tidak perlu tahu isi secrets untuk mengembangkan fitur

Tools yang perlu diinstal: Git, Go 1.25+, Node.js 24+, Docker + Docker Compose (opsional, bisa juga jalan tanpa Docker).

3. Setup Development Lokal

Ringkas dari `README.md` (baca file itu untuk detail lengkap & troubleshooting versi).

```
git clone https://github.com/Ijeon-Corp/JeonMe.git jeonme
cd jeonme

# 1. Salin file environment
cp apps/api/.env.example apps/api/.env
cp apps/web/.env.example apps/web/.env.local
# Sesuaikan JWT_SECRET di apps/api/.env (bebas isi random untuk lokal)

# 2a. Jalankan lewat Docker (paling gampang)
make up
make migrate-up

# 2b. ATAU tanpa Docker (lebih cepat untuk iterasi harian)
cd apps/api && go run main.go          # terminal 1
cd apps/web && npm install && npm run dev # terminal 2
```

Frontend: `localhost:3000` · API: `localhost:8080/api/v1` · Health: `localhost:8080/api/health`

Sebelum push, jalankan ini secara lokal (persis yang dijalankan `ci.yml`):

```
cd apps/api
go vet ./...
go test ./... -v
# opsional: golangci-lint run ./...

cd apps/web
npm run lint
npm run typecheck
npm run build
```

4. Struktur Repository

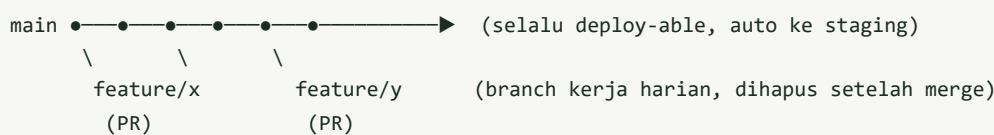
```
jeonme/
├─ apps/
│  └─ web/          # Next.js -- halaman publik + dashboard kreator
```

```
└─ api/                                     # Go/Gin -- REST API
    └─ internal/
        ├── config/                         # Baca & validasi environment variable
        ├── database/                      # Koneksi Postgres (pgxpool) & Redis
        ├── handlers/                     # HTTP handler per modul
        ├── middleware/                   # CORS, request logger, auth JWT
        ├── migrate/                     # Wrapper golang-migrate
        ├── models/                      # Struct Go untuk tabel DB
        └─ routes/                       # Pendaftaran seluruh route
└─ docker/                                # Dockerfile api & web
└─ docker-compose.yml                    # local dev
└─ docker-compose.staging.yml           # staging (VPS)
└─ docker-compose.prod.yml              # production (VPS)
└─ scripts/                             # Skrip operasional
└─ docs/                                # Dokumentasi PDF & rencana sprint
└─ .github/workflows/                   # ci.yml, deploy-staging.yml, deploy-production.yml
```

Kode di `internal/handlers` dan `apps/web/app` sering menautkan komentar ke kode requirement (`REQ-F-xxx`) dari SRS.

5. Alur Kerja Git & Kontribusi

Belum ada branch `develop` — semua kerja bercabang langsung dari `main`.



1. **Buat branch** dari main: `git checkout -b feature/nama-fitur-singkat`
2. **Commit** — pesan jelas, fokus pada *kenapa* bukan *apa*. Ikuti gaya `fix(area): ... / feat(area): ... / docs: ...` yang sudah ada di histori.
3. **Push & buka PR** ke main. CI jalan otomatis — pastikan hijau sebelum minta review.
4. **Review minimal 1 orang** → merge (squash merge disarankan).
5. Masuk main → **otomatis ter-deploy ke staging** dalam beberapa menit. Tidak ada langkah manual.
6. Staging OK & siap rilis publik → lihat Bagian 7 (butuh approval).

Yang CI cek otomatis di tiap PR:

- Backend: `go vet` , `go test` (Postgres+Redis sungguhan), `golangci-lint` , `govulncheck` (non-blocking)
- Frontend: `eslint` , `tsc --noEmit` , `next build` , `npm audit` (non-blocking)
- Image Docker dibangun & di-push ke GHCR (hanya push ke main, bukan di PR)

6. Bagaimana Staging Bekerja

Staging = <https://staging.jeonme.com> , mencerminkan main secara real-time.

```
push/merge ke main
|
```

```

▼
ci.yml (test-backend, test-frontend, build-images)
|
| sukses
|
▼
deploy-staging.yml (workflow_run, otomatis)
|
| ├── Login GHCR di VPS
| ├── Jalankan migrasi database
| ├── Pull image baru + docker compose up -d
| └── Health check (retry + verifikasi versi)
|
▼
staging.jeonme.com live dengan kode terbaru

```

Yang perlu kamu tahu sebagai developer:

- Tidak ada langkah manual — merge ke main, staging update otomatis dalam $\pm 3-5$ menit.
- Cek status: tab **Actions** di GitHub, cari run "Deploy Staging" paling atas.
- Cek versi yang sungguhan jalan: `curl https://staging.jeonme.com/api/health` — cocokkan field `version` dengan `git log` terbaru.
- Kalau Deploy Staging merah: bukan berarti kodemu salah (CI sudah lolos duluan) — biasanya masalah infra. Kabari pengelola infra.
- Staging boleh untuk eksperimen/demo, tapi datanya tidak permanen — database terpisah dari production.

7. Bagaimana Production Bekerja

Production = `https://jeonme.com` — hanya berubah lewat **tag rilis**, tidak pernah otomatis dari sekadar push ke main.

```

git tag v1.2.0 && git push origin v1.2.0
|
▼
deploy-production.yml terpicu, TERTAHAN menunggu approval
| (GitHub Environment "production" punya required reviewer)
▼
Reviewer klik "Review deployments → Approve and deploy"
|
▼
Login GHCR → migrasi → deploy → health check (verifikasi versi)
|
▼
jeonme.com live dengan rilis ini

```

Cara membuat rilis (biasanya lead/pengelola rilis):

1. Pastikan staging sudah diuji dan OK.
2. Tentukan nomor versi (semver): `v1.2.0` untuk fitur baru, `v1.2.1` untuk bugfix.
3. `git checkout main && git pull && git tag v1.2.0 && git push origin v1.2.0`
4. Tab Actions → Deploy Production menunggu "Waiting" → reviewer approve.
5. Verifikasi: `curl https://jeonme.com/api/health` — cek field `version` .

Rollback — tanpa SSH manual: di tab Actions, jalankan ulang **Deploy Production** lewat **Run workflow**, isi input `image_tag` dengan sha commit rilis sebelumnya yang sehat. Rollback image saja **tidak** menjalankan migrasi turun — kalau rilis bermasalah sudah mengubah skema database secara tidak backward-compatible, butuh keputusan manual tambahan.

8. Environment & Secrets — Ringkasan

Kamu **tidak perlu** tahu isi secrets untuk mengembangkan fitur.

- **GitHub Secrets** (per Environment staging/production): kredensial SSH ke VPS + token GHCR.
- **.env di VPS**: kredensial database, JWT_SECRET, dsb — beda nilai staging vs production, tidak pernah masuk git.
- **.env di lokalmu**: kamu isi sendiri saat setup, bebas nilainya asal valid.

Butuh environment variable baru? Update `*.env.example` (placeholder saja) dan informasikan ke pengelola infra.

9. Troubleshooting & Pertanyaan Umum

CI merah di PR-ku, kenapa?

Buka run yang gagal, expand step merah, baca errornya. Cek CICD-GUIDE.md §11 (katalog bug) — mungkin sudah pernah terjadi.

Staging tidak ter-update padahal PR-ku sudah di-merge.

Cek tab Actions — kemungkinan Deploy Staging gagal, atau masih proses. Field `version` di `/api/health` paling pasti mengonfirmasi.

Aku butuh mengubah migrasi database.

Tambah file baru di `apps/api/migrations/` (konvensi golang-migrate). Jangan edit migrasi yang sudah pernah di-deploy — selalu tambah migrasi baru.

Aku ingin tes perubahan sebelum masuk staging.

Belum ada preview environment per-PR — jalankan lokal untuk iterasi cepat, staging untuk pengujian sebelum rilis.

Aku tidak yakin fitur X sudah ada atau belum.

Cek Dokumentasi-Backend/Frontend-Jeonme.pdf bagian "Status Implementasi vs SRS", atau Rencana-Sprint-Jeonme.xlsx.

10. Peta Dokumentasi Lain

DOKUMEN	ISI	KAPAN DIBACA
README.md	Setup lokal detail, catatan versi & keamanan	Hari pertama

CICD-GUIDE.md	Arsitektur infrastruktur, isi tiap workflow, katalog bug, checklist go-live	Debug pipeline lebih dalam
SETUP-GUIDE.md	Kondisi deployment nyata + template setup VPS baru	Mengelola infra
Dokumentasi-Frontend-Jeonme.pdf	Struktur & status implementasi frontend	Kerja di apps/web
Dokumentasi-Backend-Jeonme.pdf	Struktur, routing, skema DB, status backend	Kerja di apps/api
Dokumentasi-Docker-Infrastruktur-Jeonme.pdf	Dockerfile, compose, Nginx/Apache, TLS	Kerja dengan container/deployment
Dokumentasi-CICD-Deployment-Jeonme.pdf	Ringkasan pipeline format PDF	Referensi offline/presentasi
Rencana-Sprint-Jeonme.xlsx	Backlog task per sprint, status, prioritas	Cari kerjaan / lapor progres